HOMEWORK 6 · CAPSTONE

🏦 Transaction Processing Pipeline

A file-based, AI-built pipeline: validate → detect fraud → check compliance → settle → report

Created by **Volodymyr Kiryakov**

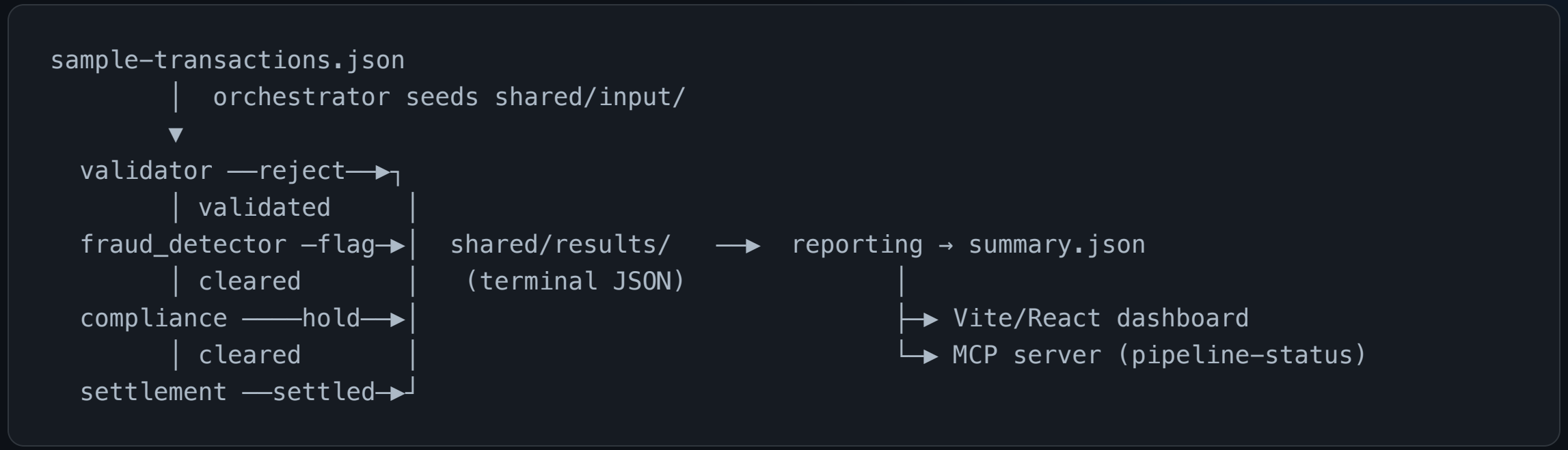
TypeScript · decimal.js · vitest · Model Context Protocol · Vite/React

What it does

Raw bank transactions flow through four stages and a reporting step. Stages exchange **JSON envelopes** through `shared/` directories — no database, no queue.

- ✔ Precise money with **decimal.js** (ROUND_HALF_UP), never floats
- ✔ Audit log per stage (ISO 8601) with **PII masking**
- ✔ Web dashboard to run & observe · custom **MCP server** to query
- ✔ **≥95%** test coverage · push blocked below 80% by a hook

Architecture — file-based envelope bus



Envelope: `{ message_id, timestamp, source_stage, target_stage, message_type, data }`. `shared/output/` is a bus keyed by `target_stage`; a generic runner wraps each pure stage handler.

Pipeline stages

STAGE	DECIDES
Validator	required fields · amount > 0 · ISO 4217 currency → else **rejected**
Fraud detector	risk score: high-value >$10k · structuring · night-time · cross-border → **flagged** at ≥50
Compliance	CTR >$10k · cross-border review · sanctions → **hold**
Settlement	fee = amount × rate · net = amount – fee (decimal) → **settled**
Reporting	aggregate results → per-status counts + reasons (summary.json)

Outcomes on the sample data (8 txns)

TXN	AMOUNT	RESULT	WHY
TXN001 / 008	1 500 / 3 200 USD	**settled**	normal
TXN004	500 EUR · 02:47 · DE	**settled**	night + cross-border noted, still valid
TXN002 / 005	25 000 / 75 000 USD	**flagged**	high-value > $10k (+ CTR)
TXN003	9 999.99 USD	**flagged**	structuring — just under $10k
TXN006	200 XYZ	**rejected**	currency not ISO 4217
TXN007	-100.00 GBP	**rejected**	amount ≤ 0

Result: 3 settled · 3 flagged · 2 rejected — every branch exercised.

The AI workflow — 4 agents

1 · Specification

specification.md + `/write-spec` skill

2 · Code generation

stages + orchestrator + UI; **context7** for MCP SDK & decimal.js

3 · Unit tests

vitest suite + **coverage-gate hook** (blocks push <80%)

4 · Documentation

README + HOWTORUN + this deck

MCP: **context7** (docs) + custom **pipeline-status** server · Skills & hook in `.claude/`

Demo

🏦 Transaction Pipeline Dashboard

▶ Run pipeline

3	3	2	8
SETTLED	FLAGGED	REJECTED	TOTAL

TRANSACTION	STATUS	REASON / FLAGS
TXN001	settled	—
TXN002	flagged	risk 60: HIGH_VALUE
TXN003	flagged	risk 55: STRUCTURING
TXN004	settled	—
TXN005	flagged	risk 60: HIGH_VALUE
TXN006	rejected	unsupported currency: XYZ (not ISO 4217)
TXN007	rejected	invalid amount: -100.00 (must be > 0)
TXN008	settled	—

Dashboard — run & observe

MCP interaction — context7 + custom pipeline-status server

Both MCP servers from mcp.json in action

```
══ context7 MCP — framework docs lookup during code generation ══

$ context7 → resolve-library-id  "Model Context Protocol TypeScript SDK"
  ✔ /modelcontextprotocol/typescript-sdk   (High reputation · 1463 snippets · score 87.6)

$ context7 → query-docs  /modelcontextprotocol/typescript-sdk
  "create McpServer with StdioServerTransport, register a tool and a resource"
  → registerTool(name, { description, inputSchema: z.object({...}) }, handler)
  → handler returns { content: [{ type: 'text', text }] }
  → registerResource(name, uri, { mimeType }, async uri => ({ contents: [...] }))
  → new StdioServerTransport(); await server.connect(transport)

$ context7 → resolve-library-id  "decimal.js"
  ✔ /mikemcl/decimal.js   (High reputation · 809 snippets · score 88.6)

$ context7 → query-docs  /mikemcl/decimal.js  "ROUND_HALF_UP for money"
  → Decimal.set({ precision: 28, rounding: Decimal.ROUND_HALF_UP })  // financial config
  → x.plus(y) / Decimal.mul(a, b) / y.toFixed(2)   → exact currency math

══ custom MCP server (pipeline-status) — live tool + resource calls ══

Tools: get_transaction_status, list_pipeline_results

> list_pipeline_results
{
  "total": 8,
  "by_status": {
    "settled": 3,
    "flagged": 3,
    "rejected": 2
  },
  "transactions": [
    {
      "transaction_id": "TXN001",
      "status": "settled"
    },
    {
      "transaction_id": "TXN002",
      "status": "flagged",
      "reason": "risk 60: HIGH_VALUE"
    },
    {
      "transaction_id": "TXN003",
      "status": "flagged",
      "reason": "risk 55: STRUCTURING"
    },
    {
      "transaction_id": "TXN004",
      "status": "settled"
    },
    {
      "transaction_id": "TXN005",
      "status": "flagged",
      "reason": "risk 60: HIGH_VALUE"
    },
    {
      "transaction_id": "TXN006",
      "status": "rejected",
      "reason": "unsupported currency: XYZ (not ISO 4217)"
    },
    {
      "transaction_id": "TXN007",
```

context7 + custom MCP calls

Lessons learned

- ✔ **Spec first paid off** — a locked envelope format made every stage a small pure function.
- ✔ **context7 beat memory** — the MCP v1.x `registerTool` API differs from older samples.
- ✔ **A generic stage runner** kept stages testable in isolation; one bus dir keyed by target.
- ✔ **The coverage gate** as a hook makes “≥80% or no push” a hard, demoable guarantee.
- ✔ **Decimal everywhere** — `0.1 + 0.2 = 0.30`, and settlement math stays exact.

Thank you — Volodymyr Kiryakov